

Cloud-native data in R

Module 1 : planetary scale data

Data Science for Environmental Policy

Today is a more thorough intro to the ideas in section of module 1:

- (review) Biodiversity data: a review querying remote tables
- (new-ish) Satellite imagery: more depth on what an image is, how to find one, and how to read part of it
- (quick look) Socio-political indicators (short)
- Guidelines for putting it all together for your Module 1 analysis

By the end of today you should be able to:

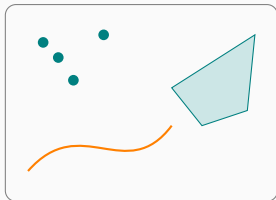
1. Explain the cloud-native (find / connect / narrow / collect) workflow and where each step runs
2. Explain what a satellite image is (bands, pixels, resolution)
3. Search a STAC catalogue for imagery and read in just the area you need

Reminder: the module

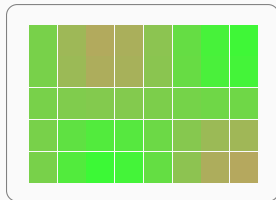
Session	Data	What you learn to query	Your task
1	GBIF occurrence records	Remote Parquet with Arrow and dplyr	A query on a taxon or place
2	Sentinel-2 imagery	A STAC catalogue, then COGs	An index for a place
3	Socio-political indicators	An indicator catalogue, then a local table	Queries on chosen indicators
4	GBIF + indicators (+ imagery)	A social or political question	Analysis and policy interpretation

Sessions 1–3 can be run on their own. Session 4 uses 1 and 3 (imagery is optional).

Reminder: types of spatial data



Vector: points, lines, polygons



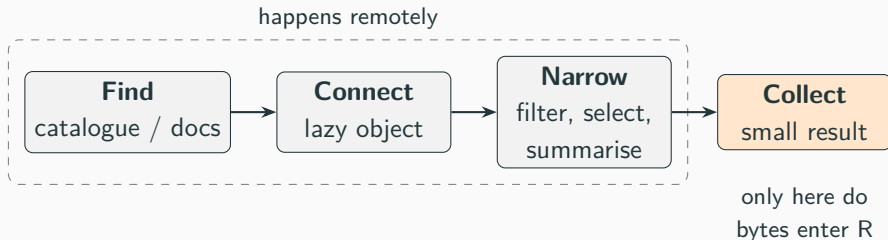
Raster: a grid of cells

iso2c	year	gdp_pc
CH	2022	77k
CR	2022	24k
KE	2022	5k

Table

	Example in this module	Cloud-native format
Vector (points)	GBIF occurrences (lat/lon columns)	Parquet
Raster	Sentinel-2 bands	Cloud Optimized GeoTIFF
Table (no geometry)	World Bank indicators, by country code	API, csv, or Parquet if large

The basic workflow



From last week:

- The usual approach: download the whole dataset, then filter it locally.
- Cloud-native approach: the data stay on the server and you only read the parts you need.
- We used the same steps for tables, rasters, and indicators.

Questions to ask for each dataset

1. Where does the dataset live?
2. How did we find the table, file, or image we need?
3. Which steps run remotely?
4. At which line does data actually come into R? ~ how big is that data?

Catalogue vs. storage

A data catalogue works like a library catalogue: it tells you what exists and where to find it. The files themselves often sit on a cloud service (e.g. AWS, Source Cooperative), but the organisation is still the data provider (e.g., NASA)

Where our three datasets live

	Where a person starts	What R connects to	R tools
GBIF	Snapshot page / AWS Open Data Registry	Parquet files on S3	arrow + dplyr
Sentinel-2	STAC catalogue (Earth Search)	STAC API, then COGs on S3	rstac + terra
World Bank	Data Catalog / WDIsearch()	World Bank API	WDI + dplyr

The formats differ, but the analysis uses the same dplyr verbs (reminder: check out a tidyverse tutorial to get oriented with these, theyll come up all semester). The extra packages are mainly for connecting to the data.

Quick detour: What is S3?

S3 (Simple Storage Service)

- Files (objects) sit in buckets, e.g. gbif-open-data-...
- Address: s3://bucket/path/to/file
- Each bucket is in a region, e.g. eu-central-1
- Public buckets don't need an account (anonymous)

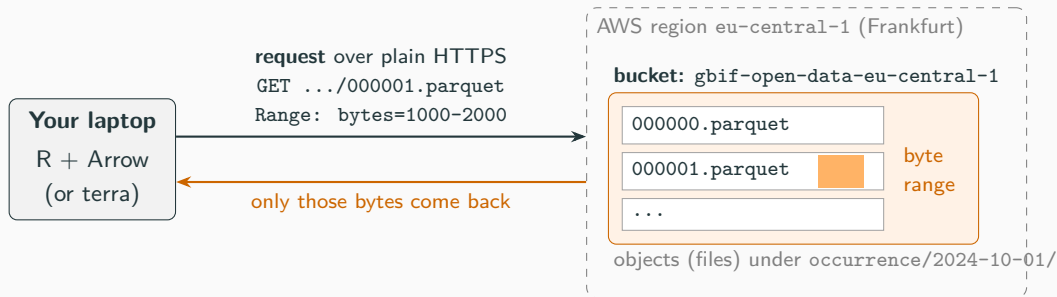
The “S3 protocol”

- Generally think of AWS S3 (Amazon's cloud file storage)
- widely copied standard so not just on AWS (MinIO, Ceph, Cloudflare R2, research clouds)
- Arrow, terra, and DuckDB all work with this standard, so the same code works on any of them

Who provides the data?

AWS hosts the files; GBIF (biodiversity section) and ESA (vegetation section) are still the data providers

How S3 works



The same requests work with other S3-compatible storage:

{MinIO} {Ceph} {Cloudflare R2} {university clouds}

1. GBIF: querying a table from afar

(Reminder) What is in GBIF?

- Mostly occurrence records: an organism observed or collected at a given place and time.
- Published by museums, universities, government agencies, research projects, and citizen science platforms.
- Billions of rows, 200+ GB, released as dated snapshots (Parquet files on S3).

Keep in mind

As discussed last week, GBIF doesn't cover all species or places equally. What ends up in the data depends on who collects and publishes records

Where GBIF records come from

Basis of record (how the record was made)

- HUMAN_OBSERVATION: e.g. eBird, iNaturalist
- PRESERVED_SPECIMEN: museum and herbarium collections
- MACHINE_OBSERVATION: camera traps, acoustic sensors
- MATERIAL_SAMPLE: e.g. DNA samples

Common biases

- Spatial: more records near roads, cities, research stations
- Taxonomic: lots of birds, far fewer insects and fungi
- Temporal: skewed towards recent years
- Institutional: depends on who has funding to digitise and publish

Several of the Session 1 queries (`count(basisofrecord)`, records per year, per kingdom) show these biases directly.

How remote reading works

Range requests

The server can send back just part of a file (e.g. bytes 1000–2000).

Columnar Parquet

Data are stored by column, so a query that uses one column only reads that column.

gbifid	species	country	year	basis

`count(countrycode)` reads only the orange column

Connect: point R at the snapshot

```
snapshot <- "2024-10-01"
```

①

```
gbif_address <- paste0(  
  "s3://anonymous@gbif-open-data-eu-central-1/occurrence/",  
  snapshot, "/occurrence.parquet?region=eu-central-1")
```

②

```
gbif <- open_dataset(gbif_address, format = "parquet")  
names(gbif)
```

③

④

- ① The snapshot date fixes the version, so results are reproducible and citable.
- ② Taken from the GBIF and AWS documentation (next slide).
- ③ Reads the file structure only, not the records (that would be massive)
- ④ Just the column names; no records downloaded yet.

Finding the snapshot address

```
s3://anonymous@gbif-open-data-eu-central-1/occurrence/
```

public bucket, no login bucket name: **AWS registry**

```
2024-10-01/occurrence.parquet?region=eu-central-1
```

snapshot date: folder of Parquet files region: **AWS registry**

GBIF snapshot page

1. gbif.org/occurrence-snapshots lists the snapshot dates and the DOI to cite.
2. registry.opendata.aws/gbif lists the bucket for each region (pick one close to you).
3. Combine the pieces into one address.

You can also list available snapshots from R:

```
b <- s3_bucket("gbif-open-data-eu-central-1",  
               region = "eu-central-1", anonymous = TRUE)  
b$ls("occurrence")      # available dates
```

Narrow, then collect

```
country_counts <- gbif |>  
  filter(!is.na(countrycode)) |>  
  count(countrycode, name = "n_records", sort = TRUE) |>  
  collect()
```

①

②

③

```
saveRDS(country_counts, "cache/country_counts.rds")  
# or save_csv(country_counts, "cache/country_counts.csv")
```

④

- ① Runs in Arrow on the remote files.
- ② Counting also runs remotely; only the countrycode column is read.
- ③ collect() is the only point where data come into R (about 250 rows, one per country).
- ④ Save the result so you don't have to rerun the query.

Tips for working with remote data

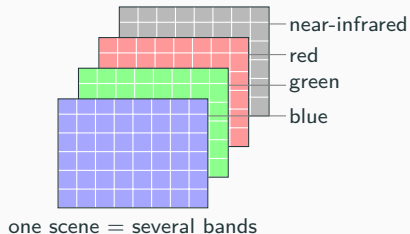
1. Use `filter()` and `select()` early so Arrow reads less data.
2. Use `count()` or `summarise()` before `collect()`.
3. After `collect()` you have a normal table you can plot, join, or save.

Don't

Don't call `collect()` on the full GBIF dataset. Why not?

2. STAC and NDVI: querying imagery from afar

What is a satellite image?



- A set of rasters, one for each band (range of wavelengths).
- Each pixel value is how much light the surface reflected in that band.
- Some bands are outside visible light, e.g. near-infrared and shortwave infrared.
- Vegetation, water, and bare ground reflect differently across bands, which is how we tell them apart.

Sentinel-2

- Part of the EU/ESA Copernicus programme; data are free and open
- 13 bands, from visible to shortwave infrared
- Resolution: 10 m (blue, green, red, NIR), 20 m (red edge, SWIR), 60 m (atmospheric bands)
- Revisits the same place about every 5 days
- Distributed as 100 × 100 km tiles (10,980 × 10,980 pixels at 10 m)
- Level-2A = corrected to surface reflectance, includes a scene classification layer (sc1)

Things to consider with any sensor

- Spatial resolution (pixel size)
- Temporal resolution (how often)
- Spectral resolution (which bands)
- Cloud cover: optical sensors can't see through clouds

These choices affect what you can and can't detect...

Finding and reading imagery: two steps

1. Search a catalogue

Find scenes that cover your area and dates with low cloud cover.

2. Read a window

Read only the pixels inside your area from the files you picked.

STAC hierarchy

Catalog: entry point (a URL)



Collection: sentinel-2-12a



Item: one scene, date, cloud %



Asset: a file, e.g. the red band

What is STAC?

STAC (SpatioTemporal Asset Catalog) is an open standard for describing geospatial data, so catalogues from different providers can be searched the same way.

Browsing in a web browser

- stacindex.org: a list of public catalogues
- STAC Browser: click through collections, items, and assets

Exploring from R

```
s <- stac(stac_url)    # Earth Search
```

```
s |> collections() |> get_request()
```

①

```
s |> collections("sentinel-2-l2a") |>  
  get_request()
```

②

```
s |> stac_search(collections =  
  "sentinel-2-l2a", limit = 1) |>  
  post_request() |> items_assets()
```

③

- ① List the collections (datasets)
- ② Details for one collection
- ③ The files (assets) in one item

Search the catalogue

```
bbox <- c(xmin = 8.45, ymin = 47.30, xmax = 8.60, ymax = 47.40)
dates <- "2023-06-01T00:00:00Z/2023-08-31T23:59:59Z"
```

①

```
search <- stac("https://earth-search.aws.element84.com/v1") |>
  stac_search(collections = "sentinel-2-l2a",
             bbox = unname(bbox), datetime = dates, limit = 100)
```

②

```
res <- post_request(search)
feats <- res$features
```

③

④

- ① Area and dates: usually the only things you need to change.
- ② Sets up the request; nothing is sent yet.
- ③ Sends the request.
- ④ Metadata for matching scenes (no pixels yet).

Pick a scene

```
info |>  
  filter(covers, cloud < max_cloud) |>  
  arrange(cloud)
```

①

②

```
item <- feats[[pick]]  
names(item$assets)
```

③

- ① Keep scenes that cover the whole area and are below the cloud threshold.
- ② Sort by cloud cover and take the first.
- ③ Files in the item: red, nir, green, scl, ...

These are the same dplyr verbs as in Session 1, used on the scene metadata.

Opening and cropping the bands

```
red <- open_asset(item, "red")
```

①

```
nir <- open_asset(item, "nir")
```

```
inMemory(red)
```

②

```
red_w <- crop(red, ext(project(aoi, crs(red))))
```

③

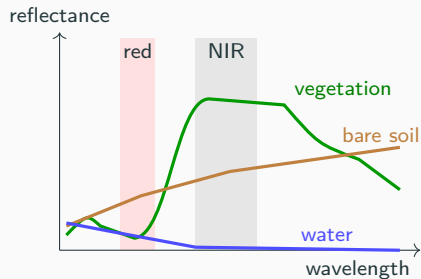
```
nir_w <- crop(nir, ext(project(aoi, crs(nir))))
```

```
c(window = ncell(red_w), tile = ncell(red))
```

④

- ① Reads only the file header (the full tile is 10,980 x 10,980 pixels).
- ② FALSE: the pixel values are still on the server.
- ③ This is where data are actually transferred. Only the COG tiles overlapping the area are read.
- ④ Compare the window with the ~120 million pixels in the full tile.

Why NDVI works



- Healthy vegetation absorbs red light (photosynthesis) and reflects a lot of near-infrared.
- NDVI compares the two:

$$\text{NDVI} = \frac{\text{NIR} - \text{Red}}{\text{NIR} + \text{Red}}$$

- Close to 1 = dense vegetation, around 0 = bare ground, negative = water.
- Other indices use different bands, e.g. NDWI (green, NIR), NBR (NIR, SWIR).

Calculate NDVI

```
ndvi <- (nir_w - red_w) / (nir_w + red_w)
scl <- open_asset(item, "scl") |> window() |>
  resample(red_w, method = "near")
ndvi_clean <- mask(ndvi, scl %in% c(4, 5, 6, 7),
  maskvalues = FALSE)
```

①

②

- ① Calculated pixel by pixel on the cropped area.
- ② Keep vegetation, bare, water, and unclassified pixels; remove cloud, shadow, and snow.

Where does the computation happen?

In this module

- Earth Search: only the catalogue
- S3: stores the files
- R on your laptop: calculates NDVI from the cropped pixels

Alternatives

- Services like openEO, Sentinel Hub, or Earth Engine can do the calculation on their servers and return the result
- This needs a different setup and workflow (happy to share resource for folks interested)

3. Socio-political indicators

```
WDIsearch("government effectiveness")
```

①

```
socio <- WDI(country = "all",  
  indicator = c(gdp_pc = "NY.GDP.PCAP.PP.KD",  
    area_km2 = "AG.LND.TOTL.K2",  
    gov_eff = "GE.EST"),  
  start = 2000, end = 2022, extra = TRUE)
```

②

① Search for indicator codes.

② One row per country per year.

Small enough that no `collect()`
is needed.

Before using an indicator

- Who produces it, and what does it actually measure? (GE.EST is based on perception surveys.)
- Which countries and years are missing?

4. Putting it together

Joining GBIF and indicators

```
latest_socio <- socio |>  
  filter(year == 2022) |>  
  select(iso2c, country, region, gdp_pc, area_km2)
```

```
national_comparison <- country_counts |> ①  
  left_join(latest_socio, by = c("countrycode" = "iso2c")) |> ②  
  filter(!is.na(area_km2), area_km2 > 0,  
         !is.na(gdp_pc), gdp_pc > 0) |> ③  
  mutate(records_per_1000_km2 = n_records / area_km2 * 1000,  
         log10_gdp_pc = log10(gdp_pc)) ④
```

- ① Country counts from Session 1.
- ② Both use ISO two-letter country codes.
- ③ Note which countries get dropped here, and report them.
- ④ Records per area, so large countries don't look better just because they're big.

What does this plot show?

```
ggplot(national_comparison,  
       aes(log10_gdp_pc, records_per_1000_km2, colour = region)) +  
  geom_point(alpha = 0.75)
```

It shows

How much countries collect, digitise, and share records through GBIF.

It doesn't show

How much biodiversity a country has, ecological condition, or monitoring quality.

A political ecology of data: the tools are part of this too

Last week we talked a lot about the data and downstream uses but the tools are part of this too

- Open Parquet files, APIs, and STAC catalogues make it easier for more people to analyse biodiversity data.
- But they also put a lot of decisions in the hands of a few institutions (GBIF, space agencies, the World Bank, cloud providers).
- Even choices like which snapshot, catalogue, or taxonomy to use are made by someone.

Module Assignment

Submit

- A clear question about one social or political dimension of GBIF data
- One GBIF summary (+ a join to a social/political indicator if useful)
- One figure or table
- Who produced each dataset you use, and your scale of analysis
- A 3–5 sentence policy interpretation
- At least two limitations of your analysis

Presentation (2–3 min, both partners)

- Question and scale
- Main pattern/result
- How it connects to one idea from the reading
- One policy implication, one limitation

Focus on your findings rather than the code!

Recap

	Find	Connect	Narrow	Collect
GBIF	snapshot docs	<code>open_dataset()</code>	<code>filter()</code> , <code>count()</code>	<code>collect()</code>
Sentinel-2	STAC search	<code>rast()</code> on a URL	<code>crop()</code> to a window	cropped pixels
World Bank	<code>WDIsearch()</code>	<code>WDI()</code> API call	<code>filter()</code> , <code>group_by()</code>	already local

And for each dataset, think about who made it and what it leaves out.

Plan for today and next week

Rest of today

- Work on the module
- Evening: discussion section on internet power

Ask questions, get help, be creative!